

Oficina AI

Relat?rio Executivo

Data: 15/09/2026

Resumo: projeto funcional localmente, com backend, frontend, banco e infraestrutura validados; estrutura pronta para orquestra??o em Kubernetes e pr?ximos passos em Kind/cluster real.

Relatório de implementação e integração — Oficina AI

****Data:**** 15/09/2026

****Objetivo:**** registrar tudo que foi configurado, validado e planejado para deixar o projeto funcional localmente e pronto para integração em orquestração.

1. Resumo executivo

Neste ciclo de trabalho, foram concluídos dois grandes blocos:

1. Correção e validação do ambiente local de desenvolvimento do projeto.
2. Integração do stack em orquestração, primeiro via ``docker-compose.yml`` e depois via Helm/Kubernetes.

Também foi registrado o estado final do backend, frontend, banco e infraestrutura, além do próximo passo de evolução: adaptar o Helm para um cluster real (AKS, EKS, GKE, Kind ou outro ambiente de produção/ambiente compartilhado).

2. Correções e ajustes realizados no ambiente local

2.1 Login e ambiente local

O login do frontend falhava por uma combinação de problemas:

- ? o banco SQLite local estava vazio, sem tabelas e sem usuário administrador;
- ? o backend não encontrava o arquivo ``.env`` corretamente quando iniciado a partir da pasta ``backend``;
- ? o cookie de sessão estava em ``Secure=True``, incompatível com HTTP local;
- ? os modelos de domínio e RBAC não estavam carregados no momento do registro de auditoria, gerando erro durante o login.

2.2 Alterações aplicadas

``backend/app/core/config.py``

- ? resolução absoluta da raiz do projeto com ``pathlib.Path``;
- ? ``.env`` agora é procurado em:
 - ? ``backend/.env``
 - ? ``.env`` na raiz do projeto
- ? ``SESSION_COOKIE_SECURE`` ajustado para ``False`` em desenvolvimento local;

? suporte adequado para execução com frontend em `http://localhost:5173`.

`backend/app/models/\_\_init\_\_.py`

? import dos modelos `Company`, `Branch`, `Permission`, `Role`, `RolePermission`, `User` e `UserRole`;

? correção do problema de `NoReferencedTableError` ao registrar o `auth.login`.

`.env` e `.env.example`

? criado `.env` local com as configurações de desenvolvimento;

? ajustado `SESSION\_COOKIE\_SECURE=false` para o ambiente local;

? documentos e instruções validados para não expor dados sensíveis em produção.

3. Banco, Redis e execução local

3.1 Banco local

Usado no ambiente de desenvolvimento:

backend/data/dev.db

Execução das migrações e seed:

Set-Location .\backend

.\venv\Scripts\python.exe -m alembic upgrade head

.\venv\Scripts\python.exe -m app.db.seed

Resultado:

? migrações aplicadas;

? empresa de exemplo criada;

? usuário administrador criado;

? login validado com sucesso.

Credenciais de dev usadas:

E-mail: admin@exemplo.com

Senha: troque-esta-senha

3.2 Redis

Foi iniciado com Docker:

```
docker compose up -d redis
```

Status validado:

PONG

4. Execução do backend e frontend

Backend

Set-Location .\backend

```
.\.venv\Scripts\python.exe -m uvicorn app.main:app --reload --port 8000
```

Validação:

GET <http://localhost:8000/api/health>

Resposta: {"status": "ok", "service": "Oficina AI"}

Frontend

Set-Location .\frontend

```
npm run dev
```

URL esperada:

<http://localhost:5173>

5. Validações executadas

5.1 Build e sintaxe

? `compileall` do backend executado com sucesso;

? `npm run build` do frontend executado com sucesso;

? login validado com HTTP 200.

5.2 Requisição real

POST <http://localhost:8000/api/auth/login>

Resposta:

```
{"status": "ok"}
```

Cookie recebido:

session_id=...

HttpOnly

SameSite=lax

6. Integração em orquestração

6.1 Docker Compose

Foi adicionado o stack de orquestração local para o projeto:

? `db` (PostgreSQL com pgvector)

? `redis`

? `backend` (FastAPI)

? `frontend` (React/Vite)

Arquivos criados/ajustados:

? [docker-compose.yml](C:/Users/MatiasFill/Desktop/oficina-ai-fase12/oficina-ai/docker-compose.yml)

? [backend/Dockerfile](C:/Users/MatiasFill/Desktop/oficina-ai-fase12/oficina-ai/backend/Dockerfile)

? [frontend/Dockerfile](C:/Users/MatiasFill/Desktop/oficina-ai-fase12/oficina-ai/frontend/Dockerfile)

? [README.md](C:/Users/MatiasFill/Desktop/oficina-ai-fase12/oficina-ai/README.md)

Status verificado:

? `docker compose config` passou sem erro de sintaxe;

? os serviços foram configurados para usar nomes de serviço na rede Docker (`db`, `redis`);

? backend e frontend foram mapeados para portas `8000` e `5173`.

6.2 Helm/Kubernetes

Como próxima evolução mais robusta, foi montado um chart Helm para o mesmo stack:

? [helm/oficina-ai/Chart.yaml](C:/Users/MatiasFill/Desktop/oficina-ai-fase12/oficina-ai/helm/oficina-ai/Chart.yaml)

? [helm/oficina-ai/values.yaml](C:/Users/MatiasFill/Desktop/oficina-ai-fase12/oficina-ai/helm/oficina-ai/values.yaml)

? [helm/oficina-ai/templates](C:/Users/MatiasFill/Desktop/oficina-ai-fase12/oficina-ai/helm/oficina-ai/templates)

O chart inclui:

? namespace;

? secret com envs;

? PostgreSQL;

? Redis;

? backend e frontend;

? ingress opcional.

Validação executada:

? render do chart com Helm concluído com sucesso;

? manifests gerados corretamente sem erro de template.

7. Arquivos principais relevantes

? [README.md](C:/Users/MatiasFill/Desktop/oficina-ai-fase12/oficina-ai/README.md)

? [docker-compose.yml](C:/Users/MatiasFill/Desktop/oficina-ai-fase12/oficina-ai/docker-compose.yml)

? [backend/app/core/config.py](C:/Users/MatiasFill/Desktop/oficina-ai-fase12/oficina-ai/backend/app/core/config.py)

? [backend/app/models/__init__.py](C:/Users/MatiasFill/Desktop/oficina-ai-fase12/oficina-ai/backend/app/models/__init__.py)

? [helm/oficina-ai](C:/Users/MatiasFill/Desktop/oficina-ai-fase12/oficina-ai/helm/oficina-ai)

8. Próximo passo planejado

O próximo passo que queremos fazer depois é:

1. adaptar o Helm para um cluster real do cliente/ambiente de execução;
2. definir `values-prod.yaml` e `values-dev.yaml`;
3. ajustar image repository, domínio, ingress, TLS e tolerâncias;
4. preparar deploy em AKS/EKS/GKE/Kind conforme necessário;
5. validar a aplicação real em cluster com health checks e conectividade aos serviços `postgres` e `redis`.

Esse passo fica registrado como a próxima mudança a ser aplicada após a base local e a base de orquestração já estarem consolidadas.

9. Estado final

Estado validado ao fim desta etapa:

- ? backend funcionando localmente;
- ? frontend funcionando localmente;
- ? login com HTTP 200 validado;
- ? Redis e SQLite funcionando para ambiente de desenvolvimento;
- ? stack integrada via Docker Compose;
- ? chart Helm criado e renderizado com sucesso;
- ? próxima etapa definida: adaptar a orquestração para um cluster real.